

# RAP 後端開發練習 1：為什麼用 RAP？環境限制聲明

## Lecture

從「寫一個 OData Service」的角度看，過去的做法有什麼問題

前面的課程已經教過幾種讓外部系統存取 SAP 資料/邏輯的方式：REST 課程 ( [src/ABAP\\_Training\\_REST/](#) ) 教的是自己寫 [CL\\_REST\\_HTTP\\_HANDLER](#) 子類手刻一個 HTTP Service；AMDP 課程教的是把運算邏輯下推到資料庫層；更早的基礎課則是傳統的 Function Module / BAPI，靠 RFC 或手動包裝暴露給外部呼叫。這幾種做法有一個共同的痛點：**CRUD (新增/查詢/修改/刪除)**、**鎖定**、**權限檢查**、**資料完整性驗證**這些「每個資料物件幾乎都要重寫一遍」的邏輯，全部要自己手動實作——自己寫 [ENQUEUE/DEQUEUE](#)、自己寫 [AUTHORITY-CHECK](#)、自己判斷這次是 Insert 還是 Update、自己維護 OData 的 [\\$metadata](#) 描述.....工程量大，而且每個開發者寫出來的風格都不一樣，維護起來很痛苦。

**RAP ( ABAP RESTful Application Programming Model )** 要解決的正是這個問題：你只要用 CDS ( Core Data Services ) 宣告資料模型、用 Behavior Definition 宣告 ( 不是用程序式邏輯寫 ) 這個資料物件支援哪些操作 ( 能不能 Create / Update / Delete )、有哪些自動衍生欄位 ( Determination )、哪些檢查規則 ( Validation )、哪些自訂操作 ( Action )，框架會自動生成鎖定管理、草稿處理 ( 如果用到 )、OData 協定轉換這些「重複性高、容易寫錯」的基礎設施代碼，開發者只需要專注在**業務邏輯**本身。這是 SAP 現在新開發案的官方建議架構，取代舊式的「Screen + FM + 手寫商業邏輯」模式 ( 俗稱 SAP GUI Programming Model )，也是 Fiori Elements 這類「畫面自動生成」前端技術背後的資料服務標準供應方式。

本課程的範圍：只教到「OData Service 正確、能被呼叫、資料進得去出得來」為止——資料模型設計、CRUD、Determinations、Validations、Actions、Service 發布。因為要銜接後續規劃中的 **Fiori Elements** 課程，rap02 會額外教 CDS View 的 **\*\*@UI.\*** Annotation 語法 ( **Metadata Extension / DDLX** 物件怎麼建、常用的 [@UI.headerInfo](#) / [@UI.lineItem](#) / [@UI.selectionField](#) 怎麼寫 ) ——但只教語法本身\*\*，List Report / Object Page 真正的版面配置邏輯、Draft 編輯體驗這些完整的畫面設計技巧，留給 Fiori Elements 課程深入。

## RAP 的五層架構



每一層各司其職：Table 只負責存資料；Interface View 負責決定資料模型長什麼樣（欄位、關聯）；Behavior Definition 負責決定這個資料物件的「行為」（能做什麼操作、有什麼規則）；Service Definition 負責決定要把哪幾個 View 打包成一個服務；Service Binding 負責決定這個服務要用哪個協定版本（OData V2/V4）真正對外發布。這五層在 rap02 ~ rap04 會逐一深入，這裡先建立整體概念。

## Managed 還是 Unmanaged？

RAP 的 Behavior Definition 有兩種實作模式：

- **Managed**：CRUD 操作的實際資料庫存取（**INSERT/UPDATE/DELETE**）由框架自動產生，你只需要宣告「這個實體支援哪些操作」，不用自己寫 **MODIFY zrapnn\_xxx FROM ...** 這種底層程式碼。新設計的資料物件，SAP 建議優先用這個模式，語法上也是官方教材的主流。
- **Unmanaged**：你自己寫一個 ABAP 類別實作所有 CRUD 邏輯（**implementation unmanaged in class ...**），框架只負責派發呼叫、不管你怎麼存資料。這個模式通常用在「要包一層 RAP 介面在既有的舊系統/BAPI 之上」的情境——這個系統裡的標準物件 **C\_SalesOrderManage**（銷售訂單管理，可以自己用 **sap-adt** 讀 **/sap/bc/adt/bo/behaviordefinitions/c\_salesordermanage/source/main** 看原始碼）就是典型例子：SD 模組的銷售訂單邏輯早就存在幾十年了（**VA01/VA02** 背後那一整套），RAP 化的時候沒有重寫，而是寫一個 **implementation unmanaged in class cl\_sd\_behv\_salesordermanage unique** 把舊邏輯包起來對外變成 RAP 服務。

⚠ **更正（rap03 出題時發現，寫這一課當下還不知道）**：這裡原本的規劃是「本課程只教 Managed，Unmanaged 太進階不教」——但 rap03 實測發現這系統的 Managed Runtime（CUD 寫入）被 SAP 官方標記為尚未對外釋出，任何 Managed BDEF 的 create/update/delete 執行到底層都會 Dump（技術細節見 rap03）。所以課程從 **rap03** 起改成兩者都教：Managed 當語法知識儲備（銜接官方教材 / 未來的 ABAP Cloud RAP 課程），Unmanaged 是這系統上真正能端對端執行、驗證的版本。Unmanaged 確實需要紮實一點的 OOP 底子（要自己實作 **LOCK/CREATE/READ** 這些方法），但既然 Managed 走不通，這是必要的取舍，詳見 rap03。

⚠ 這是 Classic RAP，不是 ABAP Cloud RAP——開課前已實測驗證的語法差異

在規劃這門課之前，我們先用一個最小的 Managed RAP BO（Table → CDS View → Managed BDEF → Service Definition，全部走 ADT / MCP 建立並成功啟用）在這個系統上做了端對端驗證，過程記錄在 [.claude/rules/sap-adt-mcp.md](#) 第 40 節。結論很清楚：這個系統的 **RAP 框架是真的在運作、不是空殼**，但屬於比較舊的世代，跟目前 SAP 官方教材 / 認證預設使用的語法有兩個具體差異：

① **沒有 View Entity 語法**：CDS 有兩種宣告根 View 的寫法。新式（2020 年之後、ABAP Cloud 世代預設）是 **define root view entity ZI\_XXX as select from ... { ... }**；這個系統用這種寫法會直接在啟用時報錯：

```
Syntax error: Keyword ENTITY not allowed
```

必須改用舊式語法（拿掉 **entity** 關鍵字）：

```
@AbapCatalog.preserveKey: true
@ObjectModel.compositionRoot: true
define root view ZI_XXX
  as select from zrapnn_xxx
{
```

```
key root_id,
descr
}
```

( 讀這個系統既有標準物件 `C_SalesOrderManage` 的原始碼可以看到，SAP 自己出的標準內容在這個系統上也是用這種舊式語法、以 `composition [0..*] of` 表達關聯，不是新式的 ON 條件簡化語法。)

② 沒有 **strict** 模式：BDEF 新式教材會在開頭寫 `strict ( 2 );` 宣告嚴格模式 ( 讓某些原本只是警告的問題變成硬性錯誤，幫助抓出設計疏漏 )。這個系統完全不認得這個語法元素：

```
managed;
strict ( 2 );      " 這一行會報 "internal | with" expected, not "strict".
```

本課程的 BDEF 一律只寫最簡單的 header：

```
managed;

define behavior for ZI_XXX alias Root
persistent table zrapnn_XXX
lock master
{
  create;
  update;
  delete;
}
```

這兩個差異背後更根本的原因，是這個系統的開發套件沒有啟用「**ABAP Cloud 語言版本**」——這是 SAP 訂出的一個語言限制開關，啟用後編譯器會擋掉一大類傳統 ABAP 寫法，目的是保證程式碼能同時跑在 On-Premise 與 SAP BTP ABAP Environment ( Cloud ) 上：

| 限制項目 | 傳統 ABAP 可以，ABAP Cloud 不行                                               |
|------|------------------------------------------------------------------------|
| 資料存取 | 只能存取已 <b>Release</b> 的 API，不能直接 <code>SELECT</code> 未 Release 的自訂表或標準表 |
| 畫面   | 沒有 Classical Dynpro，不能 <code>CALL SCREEN</code>                        |
| 程序   | 不能用 <code>FORM/PERFORM</code> ，一律用 Class/Method                        |
| 記憶體  | 不能用 <code>EXPORT/IMPORT TO MEMORY</code>                               |
| 系統存取 | 不能 <code>SET CLIENT</code> ，不能直接呼叫未 Release 的 FM                       |
| 權限   | 傳統 <code>AUTHORITY-CHECK</code> 受限，要走 Released 的權限檢查機制                 |

ABAP Cloud 語言版本要 S/4HANA **2022 以後** 的 On-Premise 系統，或 SAP BTP ABAP Environment ( Cloud ) 才能啟用——這個系統的套件沒有這個開關，所以連帶地新式 CDS View Entity / BDEF strict 語法也用不了 ( 這兩者是 ABAP Cloud 世代一起引進的語言特性 )。

⚠️ OData 服務發布：V2 能力有限，V4 完全不能透過 ADT 發布

Service Binding 決定要用哪個 OData 版本對外發布。查證這個系統的 ADT discovery 文件 ( `/sap/bc/adt/discovery` ) 發現：

- **OData V2** 有完整的發布/取消發布 API ( `/sap/bc/adt/businessservices/odatav2/publishjobs` ) ——但實測呼叫這支 API 時，回報 `Metadata not loaded ... Service Definition is not available`，這是 SAP Gateway 後端的錯誤，代表除了 ADT 層之外，系統可能還需要額外的 Gateway Hub 設定 ( 通常是 Basis 團隊在 `/IWFND/MAINT_SERVICE` 或類似交易碼做的一次性系統設定 )，不是靠寫程式能繞過的。
- **OData V4** 完全沒有對應的 ADT 發布 API——discovery 文件裡連 `odatav4` 這個 collection 都找不到 ( 雖然可以用 ADT 建立一個 V4 的 Service Binding 物件，但沒辦法讓它真正「Publish」變成可呼叫的端點 )。系統裡看得到的 V4 服務 ( 如標準物件 `C_SALESORDERMANAGE_SRV` ) 都是舊有標準內容，不是這個環境現在還能新建發布的。

這代表本課程的每一個練習，最後「服務真的能被 Postman/瀏覽器打到」這一步，需要你在 **SAP GUI** 或 **Fiori** 的 **RAP Business Services** 應用手動按 **Publish**，Claude 這邊能做到「物件建立、語法檢查、成功啟用」，跟 REST 課程 ( 第 15 節 )、Smartform 課程 ( 第 19 節 ) 的處境一樣。

OData Service 掛的路徑跟 REST 課程的自訂 SICF 服務不一樣

如果你上過 REST 課程 ( `src/ABAP_Training_REST/` )，會發現那邊每個 Service 的網址都是 `/sap/bc/zrest_training/rsNN/...`，但這門課建出來的 OData Service 網址卻是 `/sap/opu/odata/sap/<Service Binding 名稱>/...` ( V2 ) 或 `/sap/opu/odata4/sap/.../...` ( V4 ) ——兩者完全是不同的路徑根，而且建立方式的自動化程度也不一樣：

| REST 課程的自訂 SICF 服務 |                                                                                       | 這門課的 OData Service                                                                               |
|--------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| 路徑根                | <code>/sap/bc/...</code>                                                              | <code>/sap/opu/odata/sap/...</code> ( V2 )<br><code>/sap/opu/odata4/sap/...</code> ( V4 )        |
| 路徑命名               | 開發者自己取的——<br><code>zrest_training</code> 、 <code>rs01</code> 這些節點名稱是你在 SICF 手動建立時自己打的 | 框架自動產生，固定公式是<br><code>/sap/opu/odata(4)/sap/&lt;Service Binding 名稱&gt;/...</code> ，開發者沒得自己命名這段路徑 |
| 誰負責掛 SICF 節點       | 開發者自己在 SICF 手動 New Sub-Element、掛 Handler Class                                        | Service Binding 按下 <b>Publish</b> 時，框架自動幫你建立並啟用對應的 ICF 節點，不需要 ( 也不能 ) 自己去 SICF 手動建               |

為什麼要設計成這樣：`/sap/bc/` 是 ICF 底下通用、完全開放給開發者自由命名的空間，BSP 網頁、自訂 REST/HTTP Handler 都掛在這裡，SAP 不干涉你怎麼組織這段路徑。但 `/sap/opu/odata(4)/` 是 **SAP Gateway / OData Runtime** 框架保留的專用命名空間——因為 OData 服務需要讓 Fiori Launchpad、Service Catalog、`$metadata` 探索工具用統一規則去發現/呼叫任何一個服務，路徑結構必須是可預期的固定公式，不能每個開發者各自取名字，所以這段命名空間完全由框架自動管理，不開放手動掛節點。

這也是為什麼第 40.9 節那次除錯會卡住：用 ADT REST API 手動 POST 建的 Service Binding，只做到「DDIC Repository 物件存在」，沒有觸發框架把它材料化、自動掛進 `/sap/opu/odata/sap/...` 這個保留命名空間底下——這一步只有 Eclipse 官方精靈的 **Publish** 按鈕會做，這也是為什麼手動建的版本永遠 Publish 失敗、SICF 樹狀清單裡完全看不到對應節點的根本原因。

⚠ 珍貴經驗：測試網址要用 Eclipse 開出來的，不要用 SICF 「Test Service」開出來的（2026-08-02 使用者實測發現）

Publish 成功之後，Eclipse Service Binding 編輯器的 Preview... 按鈕會自動開瀏覽器，網址類似 `https://erpdemo01.itts.com.tw:44300/sap/bc/adt/businessservices/odatav2/feap?feapParams=...`——這個網址身處外網也連得到。但如果改用傳統 SICF 交易碼對某個服務節點按右鍵 Test Service，自動開出來的網址（如 `http://s4d1909fps01.itts.com.tw:50000/...`）外網通常連不到——兩者背後的原因是：

- SICF 的「Test Service」老實地把系統自己認知的內部真實主機名稱 / Port 拼進網址，這串位址通常只有內網 / VPN 環境連得到
- Eclipse 之所以能開出正確可用的外網網址，是因為 Eclipse 本身連線這套系統時，用的本來就是對外別名 `erpdemo01.itts.com.tw:44300`（Eclipse 遠端開發本來就需要外網連得到），所以它產生的所有網址自然沿用這個對外主機名稱——不是 Eclipse 這個工具比較聰明，純粹是連線來源不同

這套系統的外網存取是透過 Reverse Proxy / SAP Web Dispatcher 之類的邊界元件轉送進來的，外部看到的主機名稱 / Port 跟系統內部 ICM 實際監聽的完全是兩回事，對應關係是 Basis / 網管設定的，不會自動同步（外部通常在邊界做 SSL Termination，所以外部走 HTTPS 如 44300，內部維持單純 HTTP 如 50000）。遇到「服務明明建好啟用卻打不通」，第一件事是檢查網址裡的主機名稱是不是內部名稱，換成 `erpdemo01.itts.com.tw:44300` 這個已知對外別名，路徑部分維持不變再測一次——這招在 REST 課程（第 15/41 節）已經驗證過同樣有效，OData / REST 服務都適用。

## 跟未來課程的關係

這門課教的是能在這個系統上實際跑得動的 Classic RAP 語法——這不代表白學，資料模型設計、Managed BDEF 的 CRUD / Determination / Validation / Action 這些核心概念在 ABAP Cloud RAP 裡完全一樣，只是外層語法（`view entity` 關鍵字、`strict` 子句、ABAP Cloud 限制語法）不同。等以後有機會接上一個支援 ABAP Cloud 語言版本的系統（S/4HANA 2022+ On-Premise，或另外設定一個 SAP BTP ABAP Environment 的 MCP 連線——這是另一個獨立專案目錄的事，不會混進這個 repo），會另開一門課專門教兩者的語法差異與遷移注意事項。

## 學習目標

- 能講出 RAP 相對於「手寫 FM/BAPI/REST Handler」這幾種舊做法解決的核心問題（減少重複性基礎設施代碼）
- 能畫出 RAP 五層架構圖（Table → Interface View → Behavior Definition → Service Definition → Service Binding），講出每一層的職責
- 能分辨 Managed / Unmanaged 兩種 Behavior Definition 實作模式的差異與各自適用情境，並能在這個系統裡找到一個 Unmanaged 的標準範例（`C_SalesOrderManage`）
- 能講出這個系統跟 ABAP Cloud RAP 官方教材的兩個具體語法差異（無 `view entity`、無 `strict`），並知道背後原因是套件沒有啟用 ABAP Cloud 語言版本
- 能講出 ABAP Cloud 限制語法大致限制了哪些傳統寫法（Released API、無 Classical Dynpro、無 FORM/PERFORM 等）
- 能講出本課程 OData 服務發布的實際限制（V2 API 存在但卡在 Gateway 設定、V4 無 ADT 發布 API），知道最後一步要靠 SAP GUI 手動操作
- 知道本課程的範圍界線：後端 OData Service + CDS View 的 `@UI.*` Annotation 語法基礎（為銜接 Fiori Elements 課程鋪路），但不包含完整的 Fiori Elements 畫面設計技巧

## 下一步預告

rap02 開始動手：先設計一個簡單的資料模型 ( DDIC Table + CDS Interface View )，這是所有後續練習 ( Determination / Validation / Action / Associations ) 共同的地基；接著會加一個 Metadata Extension，教基礎的 @UI.\* Annotation 語法，幫之後的 Fiori Elements 課程鋪路。